Turtle-WoW / Tortoise-WoW Extended - Master Command Reference & Architecture Guide

This document is the canonical CLI operational reference and architecture guide for **Tortoise-WoW Extended** (`twow project/tortoise-wow`), unifying upstream VMaNGOS bugfix porting, native Turtle-WoW core restoration, deterministic bug proving, isolated worktree builds, multi-factor priority ranking, database safety auditing, and release lifecycle management.

> TIP
> **Universal Terminal Invocation** All commands can be executed from any PowerShell console or terminal without changing current directory: ```powershell & "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" <command> [arguments] [options] ```

Master Table of Contents (Index)

---

1. Overview & Operational Principles

The Tortoise-WoW porting system bridges upstream vanilla bugfixes (`vmangos/core`) with the customized Turtle-WoW 1.12.1/1.18.1 server core (`Ildourol/tortoise-wow-extended`).

Fundamental Engineering Invariants:

1. **Deterministic-First Authority**: Heuristics classify; deterministic engines prove; AI assists on ambiguity but NEVER acts as the sole validation gate.

2. **Standardized Exit Codes**:
   - `0`: PASS (Operation completed successfully and verified).
   - `1`: VALIDATION / COMPATIBILITY FAILURE (Invariant violated, collision, or check failed).
   - `2`: TOOL / ENVIRONMENT FAILURE (Missing binary, invalid repo, git error).
   - `3`: INTERNAL / SCHEMA FAILURE (Malformed contract, schema validation failure).
3. **Strict Worktree Isolation**: All candidate patches and build checks execute exclusively in dedicated Git worktrees (`.worktrees/PORT-XXXX/`). The user's primary working tree is never touched, reset, or dirtied.
4. **No Destructive Rollbacks**: `git checkout .`, `git reset --hard`, and `git clean -fd` are strictly prohibited on the target repository.
5. **No Production Database Direct Writes**: Audits analyze SQL migrations offline against cached catalog schemas (`config/schema_catalog.json`).
6. **Compile and Link Do Not Imply Startup**: Server health verifies compile, link, and startup/smoke milestones separately.
7. **Single-Writer Constraint**: Compilations through MSVC 2022 / Ninja acquire single-writer synchronization to prevent compiler file locking and git index corruption.

---

2. Architecture & Pipeline Flow

The orchestration architecture consists of three interconnected subsystems feeding through canonical state stores:

```
+----------------------------------------------------------------------------------------------------+
|                                  1. DISCOVERY & TRIAGE ENGINE                                      |
|  * Scan upstream commits (Build-CrucialRoadmap.ps1) / Forum intelligence (resources/forum/)      |
|  * Relation & Dependency Graph (RelationGraph.ps1): Supersessions, parent commits, file overlap |
|  * 12-Factor Priority Scoring Engine (PriorityEngine.ps1) -> Multi-tier candidate queue          |
+-------------------------------------------------┬--------------------------------------------------+
                                                  |
                                                  v
+----------------------------------------------------------------------------------------------------+
|                                  2. DETERMINISTIC BUG PROVER & AUDIT                               |
|  * Deterministic Bug Prover (BugProver.ps1): BUG_PRESENT, ALREADY_FIXED, NOT_APPLICABLE,          |
|    TURTLE_INTENTIONAL_DIVERGENCE, or UNCERTAIN                                                     |
|  * Compatibility Invariants (CompatibilityChecker.ps1): MAX_RACES=11, sTWDebuff, custom hooks  |
|  * Database Safety Auditor (DbAuditor.ps1): Forbidden columns (patch, build), custom ID guards   |
|    (spell_template >= 40000, world templates >= 300000), 413-table schema catalog audit           |
|  * Verification Mode Resolver (ModeEngine.ps1): Fast -> Normal -> Deep auto-escalation            |
|  * Bounded Context Assembler & AI Budgeting (AiController.ps1): Composite SHA cache               |
+-------------------------------------------------┬--------------------------------------------------+
                                                  |
                                                  v
+----------------------------------------------------------------------------------------------------+
|                                  3. ISOLATED WORKTREE BUILD & TEST                                 |
|  * Worktree Isolation (WorktreeManager.ps1): Dedicated .worktrees/PORT-XXXX/ worktree             |
|  * Build Profile Engine (BuildEngine.ps1): world, auth, sql-only, playerbots, docs-only           |
|  * Compiler Cache & CMake Generator: MSVC 2022 x64, ninja / v143 toolset                          |
|  * Disposable Server Smoke Test (SmokeTest.ps1): Startup milestone verification                   |
|  * Crash & Log Triage (TriageEngine.ps1): 17 failure categories, stack frame resolution           |
+-------------------------------------------------┬--------------------------------------------------+
                                                  |
                                                  v
+----------------------------------------------------------------------------------------------------+
|                               4. CANONICAL STATE & RELEASE MANAGER                                 |
|  * State Store (tools/state/state_store.json): 30 canonical states, atomic transitions            |
|  * Release Readiness & Manifest Generator (ReleaseManager.ps1): docs/releases/*.json            |
+----------------------------------------------------------------------------------------------------+
```

---

3. Workflow Lifecycle of a Candidate Fix

```
[Upstream Commit]
       │
       ▼
1. DISCOVERED ──> 2. TRIAGED ──> 3. BUG_PROOF_PENDING
                                        │
                    ┌───────────────────┴───────────────────┐
                    ▼                                       ▼
```

```
           [ALREADY_FIXED] /                              [BUG_PRESENT]
           [NOT_APPLICABLE] /                                   │
           [TURTLE_DIVERGENCE]                                  ▼
                   │                                  4. DEPENDENCY_CHECK
                   │                                            │
                   ▼                                            ▼
              (REJECTED)                               5. DB_MIGRATION_CHECK
                                                                │
                                                                ▼
                                                       6. WORKTREE_CREATION
                                                                │
                                                                ▼
                                                       7. COMPILE_VERIFICATION
                                                                │
                                                                ▼
                                                       8. STARTUP_SMOKE_TEST
                                                                │
                                                                ▼
                                                       9. CANDIDATE_BRANCH_COMMIT
                                                                │
                                                                ▼
                                                      10. STATE_STORE_COMPLETE
```

---

4. Master Command Reference: Core Automation Commands

Command	Full Syntax	Mode	Access	AI Usage	Build Usage	DB Usage	Description		
`task auto-pilot`	`task.ps1 auto-pilot [N] [-Tier 1-5] [-Mode M]`	Variable	Write & Remote Push	Advisory	Full MSVC	Migration Audit	Autonomous batch port, compile, candidate commit, and auto-push passing fixes to GitHub `extended` branch.		
`task auto-port`	`task.ps1 auto-port <sha> [-Mode M] [-DryRun]`	Variable	Write & Remote Push	Advisory	Full MSVC	Migration Audit	Autonomous single-commit port, compile, candidate commit, and auto-push to GitHub `extended` branch.		
`task push-extended`	`task.ps1 push-extended [branch] [-DryRun]`	Fast	Remote Git Push	None	None	None	Integrates and pushes all verified passing candidate commits to remote `extended` branch (`extended/extended`).		
`task port`	`task.ps1 port <sha> [-Mode Fast\	Normal\	Deep] [-DryRun] [-AutoCommit]`	Variable	Write (Worktree)	Advisory	Patch-Aware	Migration Audit	Unified porting pipeline for upstream donor commits (push requires `-AutoCommit`).
`task port-batch`	`task.ps1 port-batch <N> [-Tier 1-5] [-Mode M] [-DryRun] [-AutoCommit]`	Variable	Write (Worktree)	Advisory	Patch-Aware	Migration Audit	Batch executes porting pipeline on next N curated candidate commits (push requires `-AutoCommit`).		
`task build`	`task.ps1 build <profile>`	Fast/Deep	Execute	None	Full MSVC	None	Builds server profile (`world` , `auth` , `sql-only` , `playerbots`).		
`task build-packages`	`task.ps1 build-packages`	Normal	Write (Worktree)	None	Full MSVC	Offline Audit	Compiles, verifies, and commits all staged packages in worktrees (local only, never pushes).		
`task 2`	`task.ps1 2 <sha/topic>`	Fast	Read-only	None	None	None	Searches 22,155 indexed forum threads for bug discussions and mechanics lore.		
`task 3`	`task.ps1 3 [tbl] [id]`	Fast	Read-only	None	None	Brotalnia / Base / Viewer	Audits pending database migration SQL or scalps table entity details.		

Command	Full Syntax	Mode	Access	AI Usage	Build Usage	DB Usage	Description
`task 4`	`task.ps1 4 <sha>`	Normal	Read-only	Advisory	None	None	Generates bounded AI semantic dossier with touched code and context lines.
`task 5`	`task.ps1 5 <topic>`	Normal	Read-only	Advisory	None	Offline Catalog	Native Turtle core specification auditor for missing custom features.
`task 6`	`task.ps1 6 <id/query>`	Fast	Read-only	None	None	tortoise-db-viewer API	Queries official Turtle Online DB viewer for item, spell, and creature tooltips.
`task scalp`	`task.ps1 scalp <tbl> <id> [-Diff] [-Export]`	Fast	Read-only	None	None	Brotalnia / Base / Viewer	Extracts entity definitions and generates side-by-side vanilla vs Turtle diffs.
`task extract`	`task.ps1 extract <tbl> <id>`	Fast	Read-only	None	None	Brotalnia / Base / Viewer	Alias for `task scalp`.
`task dashboard`	`task.ps1 dashboard [id]`	Fast	Read-only	None	None	Web Dashboard	Launches official AoWoW-style web dashboard in browser (alias: `task viewer`).
`task restore`	`task.ps1 restore <topic> [-StageTemplate]`	Normal	Staging	Advisory	None	Offline Catalog	Audits official staff posts and stages native core restoration manifests.
`task restore-batch`	`task.ps1 restore-batch <N>`	Normal	Staging	Advisory	None	Offline Catalog	Batch audits next N un-audited Turtle patch topics from roadmap queue.
`task status`	`task.ps1 status`	Fast	Read-only	None	None	None	Displays live system metrics, queue counts, HEAD SHAs, and active run IDs.
`task pdf`	`task.ps1 pdf`	Fast	Read-only	None	None	None	Compiles `COMMAND_REFERENCE.md` to HTML and exports high-quality PDF via Edge.

5. Master Command Reference: Extended Engineering Commands

Command	Full Syntax	Mode	Access	AI Usage	Build Usage	DB Usage	Description	
`task system-check`	`task.ps1 system-check [light\	full]`	Fast / Deep	Read-only	Internal Gate	Preflight / 39-Tests	Full Catalog	Pre-flight audit: verifies docs, locations, goals, instructions, configs, worktrees, compilers, and test suite.
`task test`	`task.ps1 test`	Fast	Read-only	None	None	None	Executes comprehensive 39-case Pester test suite covering all invariants.	
`task prove`	`task.ps1 prove <sha>`	Fast	Read-only	None	None	None	Deterministic bug prover inspecting diff hunks and target AST existence.	
`task relations`	`task.ps1 relations <sha>`	Fast	Read-only	None	None	None	Analyzes commit reverts, duplicate fixes, and file overlap relationships.	
`task dependencies`	`task.ps1 dependencies <sha>`	Fast	Read-only	None	None	None	Inspects git commit DAG for missing parents and prerequisite commits.	
`task rank`	`task.ps1 rank`	Fast	Read-only	None	None	None	Computes 12-factor priority scores (0.0–100.0) across all pending candidates.	
`task next`	`task.ps1 next`	Fast	Read-only	None	None	None	Returns the single highest-priority eligible candidate commit to port.	
`task plan`	`task.ps1 plan <sha> [-Mode M]`	Variable	Read-only	None	None	None	Generates a complete verification plan and dry-run report without disk writes.	

Command	Full Syntax	Mode	Access	AI Usage	Build Usage	DB Usage	Description
`task build-profile`	`task.ps1 build-profile <sha>`	Fast	Read-only	None	None	None	Selects optimal build target (`world`, `auth`, `sql-only`, `playerbots`).
`task smoke`	`task.ps1 smoke [-Mode M]`	Fast/Deep	Execute	None	Binary Test	None	Disposable server startup smoke test evaluating `mangosd.exe` & `realmd.exe`.
`task crash`	`task.ps1 crash <path>`	Fast	Read-only	None	None	None	Parses server crash dump / log, categorizes failure, and extracts stack frames.
`task triage`	`task.ps1 triage <logfile>`	Fast	Read-only	None	None	None	Categorizes log errors across 17 structured failure categories.
`task baseline`	`task.ps1 baseline [-Force]`	Fast	Read-only	None	Binary Test	None	Checks and caches compile, link, and startup health of target repository HEAD.
`task state`	`task.ps1 state`	Fast	Read-only	None	None	None	Outputs canonical pipeline state store summary and active run statistics.
`task config-check`	`task.ps1 config-check`	Fast	Read-only	None	None	None	Validates repository paths, toolchain binaries, and schema configuration.
`task state-check`	`task.ps1 state-check`	Fast	Read-only	None	None	None	Verifies state store integrity, active runs, and candidate transition history.
`task worktree`	`task.ps1 worktree`	Fast	Read-only	None	None	None	Lists all currently active and managed isolated Git worktrees.
`task worktree-cleanup`	`task.ps1 worktree-cleanup [-DryRun]`	Fast	Write (Worktree)	None	None	None	Safely purges obsolete or completed worktrees inside `.worktrees/`.
`task parity`	`task.ps1 parity`	Fast	Read-only	None	None	Offline DBC	Audits binary client DBCs (`ChrRaces`, `Map`, `Spell`) against server code.
`task release-check`	`task.ps1 release-check`	Fast	Read-only	None	None	Offline Catalog	Evaluates all release readiness gates (tree clean, invariants pass, baseline).
`task release-manifest`	`task.ps1 release-manifest [name]`	Fast	Write (Docs)	None	None	None	Generates a signed machine-readable release manifest in `docs/releases/`.
`task tag-release`	`task.ps1 tag-release <name>`	Fast	Write (Git Tag)	None	None	None	Gated Git tag creation; rejects release if any readiness gates fail.
`task roadmap-refresh`	`task.ps1 roadmap-refresh [-FetchLatest]`	Fast	Read/Write (Roadmap)	None	None	None	Audits all 7,300+ upstream commits, refreshes queue, and fetches latest commits.

6. Verification Modes: FAST vs NORMAL vs DEEP

The system provides three strictly defined verification modes. Each mode enforces all safety invariants, but varies in compilation scope, runtime validation depth, and AI assistance:

Verification Mode	Code Analysis	Invariant Scan	DB Schema Audit	Build Execution	Startup Smoke Test	Runtime Test	AI Consultation	Typical Duration
Fast	Deterministic diff matching	Full manifest invariant scan	Full 413-table column audit	Skipped	Skipped	Skipped	Skipped (Deterministic only)	1 – 3 seconds
Normal	Diff + Nearby context AST	Full manifest invariant scan	Full 413-table column audit	Patch-Aware (`world` / `auth`)	Disposable startup check	Optional / Smoke	Advisory (On ambiguity only)	30 – 90 seconds
Deep	Full call-graph & packet trace	Full manifest invariant scan	Full 413-table column audit	Full MSVC Solution	Full disposable smoke test	Reproducer verification	Advisory + Contextual trace	2 – 5 minutes

Mandatory Common Checks (Enforced Across ALL Modes):

1. **Deterministic Bug-Existence Proof**: Verification that the defect exists in target and was not previously fixed or intentionally diverged.

2. **Compatibility Invariant Scan**: Verifies `MAX_RACES = 11`, `sTWDebuff`, `SCRIPT_COMMAND_TAKE_MONEY = 93`, and protected manager hooks.

3. **Database Migration Safety Check**: Forbidden progressive versioning columns (`patch`, `build`) and custom ID boundaries (`spell_template` >= 40000, world templates >= 300000).

4. **Target Working Tree Cleanliness**: Main checkout is verified clean before worktree creation.

---

7. Automatic Mode Escalation Rules

The mode engine (`ModeEngine.ps1`) evaluates candidate risk factors. It automatically escalates execution modes to protect server integrity:

Fast $\rightarrow$ Normal Escalation Triggers:

- Candidate introduces or modifies SQL database migration files (`sql/database_updates/`).
- Candidate possesses one or more unresolved predecessor commit dependencies.
- Bug-existence confidence score is below 0.85.
- Candidate modifies core spell mechanics (`SpellMgr.cpp`, `SpellEffects.cpp`).

Normal $\rightarrow$ Deep Escalation Triggers:

- Commit subject or diff touches security keywords: `crash`, `packet`, `opcode`, `thread`, `mutex`, `deadlock`, `leak`, `exploit`, `auth`, `crypto`.
- Touched files reside in security or networking subsystems (`src/shared/Auth/`, `src/game/WorldSocket.cpp`, `src/game/Opcodes.cpp`).
- Concurrency constructs are modified (locks, atomic variables, threading queues).

> **IMPORTANT**
> **No Automatic Downgrade Policy** Under no circumstances will an explicitly requested mode be automatically downgraded (e.g., `-Mode Deep` will never execute as `Normal` or `Fast`).

---

8. DryRun Mode & Safety Guarantees

Running any porting or cleanup command with `-DryRun` guarantees that **zero modifications** are written to the target repository or filesystem:

```
# Plan candidate 448df9ba0 without touching repository
task.ps1 port 448df9ba0 -DryRun
```

DryRun Guarantees:

- No Git branches created (`port/PORT-XXXX` is not created).
- No Git worktrees spawned (`.worktrees/` remains unchanged).
- No compiler or linker invocations executed.
- No database files modified.
- Outputs complete execution plan: Bug verdict, priority score, resolved mode, required build profile, and dependency links.

---

9. Auto-Pilot Mode & Autonomous Batch Processing

Auto-Pilot Mode enables hands-free, continuous candidate evaluation, bug proving, worktree provisioning, patch normalization, compatibility auditing, and candidate staging across multiple commits without manual per-commit intervention.

The One-Command End-to-End Pipeline

When you want an autonomous, zero-friction pipeline that takes upstream commits all the way to candidate branch commits in a single pass, use **Auto-Pilot Mode**:

```
# Auto-Pilot a batch of 10 candidates with automatic compilation and git commit:
task.ps1 auto-pilot 10

# Auto-Pilot with tier filter:
task.ps1 auto-pilot 10 -Tier 1
task.ps1 auto-pilot 10 1

# Auto-Pilot a specific single commit:
task.ps1 auto-port 448df9ba0
```

How the Chained Pipeline Works:

1. **Forum & Mechanics Intelligence Scout (`Search-ForumArchive.ps1`)**:

Automatically mines 22,155 indexed Turtle-WoW forum threads using keywords extracted from the commit subject. Identifies any related mechanics discussions, player bug reports, or staff statements.

2. **Database & Migration Safety Audit (`Audit-DatabaseMigrations.ps1`)**:

Checks whether the upstream commit touches SQL migrations. Inspects table definitions against the 413-table schema catalog, verifies Turtle custom ID ranges (creature >= 300,000, spell >= 40,000), checks for forbidden progressive columns, and stages any required SQL files into `tools/queue/staging_sql/`.

3. **AI Semantic Conflict & Regression Audit Gate (`AiConflictAuditor.ps1`)**:

Replaces shallow static text searches with deep contextual AI semantic auditing across six core dimensions:

- *Race Invariant*: Verifies bounded loops/arrays support Turtle's 11 races (`MAX_RACES = 11`, High Elf & Goblin).
- *Debuff Streaming*: Guarantees 64-bit `sTWDebuff` streaming is never truncated by 32-bit donor primitives.
- *Manager Integrity*: Verifies protected managers (`sLFTMgr`, `sTransmogMgr`, `sCustomMerchantMgr`) remain untouched.
- *Entity Protection*: Strictly guards custom ranges (spells ≥ 40000, entities ≥ 300000).
- *Call-Site Signatures*: Validates that donor call-sites retain custom parameters (e.g. `inGurubashiArena`).
- *Concurrency Safety*: Evaluates lock hierarchies (`m_mapLock`, `m_objectLock`) for AB-BA deadlock prevention.

If any semantic conflict is detected, the candidate is automatically rejected with `AI_SEMANTIC_CONFLICT` before staging or compiling.

4. **AI Context Assembly & Semantic Dossier (`Invoke-AiAudit.ps1`)**:

Performs bounded diff context extraction, checks surrounding code ASTs, evaluates Turtle custom divergences, generates an AI audit dossier in `tools/queue/ai_dossiers/<sha>.md`, and packages the candidate into `tools/queue/02_ready_to_build/PORT-XXXX.json`.

5. **Smart Path Mapping & Directory Normalization (`PathMapper.ps1`)**:

Dynamically and statically maps upstream donor file paths into the canonical Tortoise-WoW directory layout (e.g., `src/scripts/eastern_kingdoms/<zone>/<dungeon>/` $\rightarrow$ `src/scripts/dungeons/<dungeon>/`, `contrib/<tool>/` $\rightarrow$ `tools/<tool>/`, and `cmake/find/` $\rightarrow$ `cmake/`) across 519+ tracked reorganizations so that patch application targets the correct files with zero path corruption.

6. **Isolated Worktree Builder & Committer (`Build-ReadyPackages.ps1`)**:

Creates an isolated git worktree (`.worktrees/candidate-PORT-XXXX`), applies the normalized patch safely, enforces build profiles (`world`, `auth`, `sql-only`), compiles via MSVC 2022, and commits to a candidate branch with full provenance recorded in `tools/state/state_store.json`.

7. **Remote Extended Branch Integration & Push (`Push-PassingCandidates.ps1`)**:

Automatically aggregates all certified passing candidate commits and pushes them to the development branch: `https://github.com/Ildourol/tortoise-wow-extended/tree/extended` (`extended/extended`).

FAQ: Staging Mode vs Auto-Pilot Mode

Execution Command	Staged to Queue?	Automatically Compiled?	Automatically Committed?	Pushed to GitHub `extended` ?
`task.ps1 port-batch 10`	Yes (`02_ready_to_build/`)	No	No	**NO** (Staging only)
`task.ps1 build-packages`	Pre-staged	Yes (in worktree)	Yes (candidate branch)	**NO** (Local worktrees only)
`task.ps1 auto-pilot 10`	Yes	Yes (in worktree)	Yes (candidate branch)	**YES** (Pushed to `extended/extended`)
`task.ps1 port-batch 10 -AutoCommit`	Yes	Yes (in worktree)	Yes (candidate branch)	**YES** (Pushed to `extended/extended`)
`task.ps1 auto-port <sha>`	Yes	Yes (in worktree)	Yes (candidate branch)	**YES** (Pushed to `extended/extended`)
`task.ps1 push-extended [branch]`	Pre-committed	N/A	Integrates to branch	**YES** (Pushed to `extended/extended`)

> **IMPORTANT**
> **Strict Push Safety Guarantee**: Remote git pushes are strictly gated: pushing will **ONLY** happen when running `auto-pilot`, `auto-port`, when `-AutoCommit` is explicitly provided, or via `task push-extended`. Standard staging commands (`task port`, `task port-batch`), manual local builds (`task build-packages`), and dry runs will **NEVER** push to remote.

Clean CLI Invocations (From Default Open Location / Any Directory)

You do **not** need to `cd` into the project repository. All modules dynamically resolve repository roots from `$PSScriptRoot`.

Direct Invocation from PowerShell (from `C:\Users\Admin>` or any path):

```
& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" auto-pilot 10
```

3. Permanent Global Setup via PowerShell `$PROFILE` (Recommended):

Run this once in PowerShell to make `task` globally accessible from any terminal:

```
if (!(Test-Path $PROFILE)) { New-Item -ItemType File -Path $PROFILE -Force | Out-Null }
Add-Content $PROFILE "`nfunction task { & 'C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1' @ar
```

Once added, open a new PowerShell prompt anywhere and simply type:

```
task auto-pilot 10
task status
task next
task roadmap-refresh
```

Running Auto-Pilot Mode: With Tier vs Without Tier

A. Running WITH Tier Specification:

When you want to focus exclusively on a specific severity or architectural domain:

```
# Port the next 10 Tier 1 candidates (Crashes, Leaks, Deadlocks, Exploits)
task.ps1 port-batch 10 -Tier 1 -Mode Normal

# Port the next 10 Tier 2 candidates (Combat Accuracy, Spells, Formulas)
task.ps1 port-batch 10 -Tier 2 -Mode Normal
```

B. Running WITHOUT Tier Specification:

When you omit the `-Tier` argument:

```
# Auto-Pilot without tier: ports the next 10 highest-value candidates
task.ps1 port-batch 10 -Mode Normal
```

How Does Auto-Pilot Choose Commits Without a Tier?

1. **Natural Architectural Hierarchy in `CRUCIAL_COMMITS_QUEUE.csv`**:

The master queue is pre-sorted by architectural urgency: **Tier 1 (Crashes/Exploits)** $\rightarrow$ **Tier 2 (Combat/Spells)** $\rightarrow$ **Tier 3 (NPC/AI)** $\rightarrow$ **Tier 4 (Movement/Maps)** $\rightarrow$ **Tier 5 (General)**. Without a tier filter, Auto-Pilot processes strictly down this natural hierarchy.

2. **Dynamic 12-Factor Priority Scoring Engine (`task rank` / `task next`)**:

In addition to tier order, every candidate is scored dynamically from 0.0 to 100.0 by `PriorityEngine.ps1`:

- Crash Severity: $+25$ points
- High Player Impact: $+20$ points
- Security / Exploit Risk: $+20$ points
- Low Target File Churn: $+15$ points
- Dependency Satisfaction: $+10$ points
- Baseline Stability: $+10$ points
- Non-Applicable / Intentional Divergence: **0.0 score hard gate** (instantly skipped).

You can inspect this ranking at any time with `task rank` or retrieve the single best candidate with `task next`.

3. **State Store & Git History Deduplication**:

Auto-Pilot automatically cross-references `tools/state/state_store.json` and the target `tortoise-wow` git history. Any candidate that has already been evaluated (`ALREADY_FIXED`, `NOT_APPLICABLE`, `TURTLE_INTENTIONAL_DIVERGENCE`, `PACKAGE_READY`, or `RELEASED`) is **automatically bypassed**, ensuring zero redundant work.

The Recommended 3-Phase Auto-Pilot Runbook

To run Auto-Pilot with 100% safety and predictability:

```
# Phase 1: Pre-Flight DryRun (Evaluates bug prover and outputs plan without writing files)
task.ps1 port-batch 10 -DryRun

# Phase 2: Isolated Autonomous Porting (Runs in .worktrees/PORT-XXXX/ with safety checks)
task.ps1 port-batch 10 -Mode Normal

# Phase 3: Single-Writer Candidate Build & Branch Commit
task.ps1 1
```

Auto-Pilot Safety Guarantees:

- **Zero Working Tree Contamination**: All patch application and test compilation occur strictly in `.worktrees/PORT-XXXX/`. The main repository working tree remains 100% untouched.
- **Stale Package Auto-Invalidation**: If the target repository HEAD advances during a batch run, any staged package whose pinned base SHA does not match is automatically invalidated.
- **Non-Destructive Rollback**: If a patch fails compilation or invariant checks, the isolated worktree is safely deleted without ever executing `git reset --hard` or `git checkout .`.

---

10. Roadmap Research, Commit Auditing & Catching New Upstream Commits (`task roadmap-refresh`)

The upstream VMaNGOS repository (`vmangos/core`) continuously accepts new bugfixes and accuracy patches. The orchestration framework includes an automated research and auditing engine to catch, classify, and queue these new commits:

```
# Audit all local upstream commits and regenerate the roadmap
task.ps1 roadmap-refresh

# Fetch new commits from GitHub upstream remote, merge development, and audit
task.ps1 roadmap-refresh -FetchLatest
```

How the Roadmap List Refreshes:

1. **Upstream Remote Fetch (`-FetchLatest`)**: When `-FetchLatest` is passed, the engine runs `git fetch origin` and `git merge --ff-only origin/development` inside `reference-upstreams/vmangos-core`, pulling down all new upstream commits.
2. **Donor Commit Scanning**: Scans all 7,300+ upstream commits in repository history (newest to oldest).
3. **Cross-Referencing Port History**: Compares every upstream commit SHA against commits already merged into `tortoise-wow` and documented in `docs/BACKPORT_HISTORY.md`.
4. **Older Superseded Duplicate Elimination**: Detects intermediate or superseded fixes and removes older duplicates so only modern final fixes are queued.
5. **Architectural Tier Classification**: Automatically assigns each commit to Tier 1 through Tier 5 based on modified subsystems and commit subjects.
6. **Artifact Generation**:
   - Updates `tools/porting/CRUCIAL_COMMITS_QUEUE.csv` (the active queue of 5,092 deduplicated crucial candidates).
   - Updates `tools/porting/ALL_AVAILABLE_COMMITS_REFERENCE.csv` (complete 7,339-commit catalog).
   - Re-renders `docs/ROADMAP.md` with updated metrics and live backlog status.

---

11. Build Profiles & Compiler Toolchain

The build engine (`BuildEngine.ps1`) optimizes build times by targeting only the solution components touched by the candidate patch:

Build Profile	Affected Subsystems	MSVC Target Project	Typical Compile Time
`world`	`src/game/`, `src/scripts/`	`mangosd`	~45 seconds
`auth`	`src/realmd/`, `src/shared/Auth/`	`realmd`	~15 seconds
`sql-only`	`sql/`, documentation	None (Bypasses compilation)	Instant (0s)
`playerbots`	`src/game/playerbot/`	`mangosd` (PlayerBots profile)	~45 seconds
`docs-only`	`docs/`, `tools/`	None	Instant (0s)

Build & Verification Execution Modes & Token Optimizations

Compiling and linking `mangosd.exe` on Windows requires 2–3 minutes and generates massive MSVC output that causes rapid context token bloat. The pipeline supports three execution strategies and five high-efficiency execution optimizations:

Five High-Efficiency Execution Optimizations:

1. **Dynamic CPU Parallelism (`--parallel $env:NUMBER_OF_PROCESSORS`)**: Uses all 12 CPU cores dynamically, accelerating parallel C++ compilation by 2.5x–3x.
2. **MSBuild Quiet Verbosity (`/nologo /v:q` / `/v:m`)**: Passing compilation emits zero progress lines into context, preventing thousands of tokens of context bloat. Diagnostics print only on errors.
3. **Fast Static Library Targeting (`-FastBuild`)**: Enables `task 1 -FastBuild` / `task build-ready -FastBuild` to verify only affected static libraries (`game.lib`, `modules.lib`, `shared.lib`) in ~3 seconds.
4. **Concise Git Operations (`--quiet`)**: Routine git operations run with `-q` to preserve context window cleanliness.
5. **Preserved Atomic Git Provenance**: Every patch produces an isolated, atomic commit on candidate branch `port/PORT-XXXX` with full 1-to-1 donor attribution and dossier.

Execution Mode	Per-Candidate Step	Git Commit & Push	Milestone / Batch Gate	Token Impact	Build Latency	Best Used For
Option 1: Fast Incremental (Default)	Compile affected static library (`game.lib` / `modules.lib`, ~3s) via `-FastBuild`	Atomic commit & remote push	Link `mangosd.exe` once at batch end	~95% token reduction	Fast (~3s/commit)	Routine daily development and multi-commit batches
Option 2: Batch Verification	Sequential git commit with atomic dossiers	Commit sequentially	Full compile (`world`) + link (`mangosd.exe`)	Lowest token cost (1 pass total)	Fastest	High-throughput backlog clearance
Option 3: Strict Full-Link	Full compile and link (`mangosd.exe`) per candidate	Commit & push 1-by-1	Verified individually on every commit	High (hundreds of lines/commit)	Slow (~2-3m/commit)	P0 critical fixes, threading, mutexes, memory

Debugging & Error Isolation Guarantees:

- **Compiler Errors**: MSVC output explicitly identifies the source file, class/function, and exact line number. When a batch compile fails, the compiler error immediately identifies which specific commit in the batch is responsible.
- **Runtime Bugs & Bisectability**: Because candidate code changes are committed as individual atomic commits in git, standard `git bisect` functions identically regardless of whether the build was verified incrementally or in batch.
- **Binary Identity**: The resulting binaries produced by all three options are identical bit-for-bit.

---

12. System Pre-Flight & Health Audit (`task system-check`)

Before operators, engineers, or automated agents begin development, porting, or building, the **System Pre-Flight Audit** (`tools/modules/SystemChecker.ps1`) verifies total project health across 6 core operational layers in two tailored modes:

```
# 1. Light System Check (Default: ~1.7s fast pre-flight verification)
task.ps1 system-check
task.ps1 system-check light

# 2. Full System Check (Deep pre-flight audit with compiler & 39-test suite: ~10.5s)
task.ps1 system-check full
```

Audit Modes Compared

Feature / Inspection Dimension	Light Mode (`task system-check light`)	Full Mode (`task system-check full`)
Execution Duration	**~1.7 seconds**	**~10.5 seconds**
1. Repository Locations	Project root, `tortoise-wow` , `vmangos-core` , client-data, forum	Full repository layout verification
2. Pipeline Staging Queues	7 queue directories verified & initialized	7 queue directories verified & initialized
3. Documentation Suite	12 core documentation files checked (size & presence)	12 core documentation files checked (size & presence)
4. Project Goals & Invariants	`MAX_RACES = 11` , `sTWDebuff` , opcode `93` , 5 protected symbols	Invariant policies + mock patch gate violation dry-run
5. Authority Hierarchy	9-tier hierarchy and 3 forbidden actions verified	Full authority hierarchy validation
6. Agent Instructions	All 6 agent prompt runbooks in `tools/tasks/` verified	All 6 agent prompt runbooks in `tools/tasks/` verified
7. Configuration & State Store	`twow-project.json` , 413-table catalog, state store schema	Full configuration discovery & state store parsing
8. Git Tree & Worktrees	Main working tree cleanliness, 0 dangling worktrees	Working tree cleanliness, active worktrees audit
9. Build Toolchain Preflight	Skipped	Git, CMake 4.4+, MSVC 2022 / v143 toolset detection
10. Remote Synchronization	Skipped	Verifies `origin` and `extended` remotes configured
11. AI Semantic Auditor	Skipped	6-dimension AI conflict auditor readiness check
12. Full Orchestration Tests	Skipped	Complete 39-test Pester test suite execution (100% pass)
Target Audience	Routine daily pre-flight / quick verification	Clean checkout onboarding, pre-release certification

Structured Exit Codes:

- **`Exit Code 0`** : `FULLY CERTIFIED & READY TO OPERATE` (All checks passed).
- **`Exit Code 1`** : `BLOCKED - RESOLVE FAILURES BEFORE OPERATING` (One or more critical blockers detected).

---

13. Baseline Health Check & SHA Pinning

The baseline checker (`BaselineChecker.ps1`) verifies the compile, link, and startup state of the unchanged target repository before applying candidate patches:

```
task.ps1 baseline
```

- **Cached Baseline**: Once verified for a target HEAD SHA, baseline status is cached in `tools/state/state_store.json` .
- **Three-Tier Classification**:
- `COMPILE_FAIL` : Target source fails compilation prior to patch.
- `LINK_FAIL` : Compilation passes, but linker errors occur.
- `STARTUP_FAIL` : Binary links, but crashes on startup (e.g. missing DBC or DB config).
- `HEALTHY` : Compile, link, and startup all succeed.

14. Bug-Existence Proving Engine

The bug prover (`BugProver.ps1`) executes deterministic analysis before any candidate is ported:

```
task.ps1 prove 448df9ba0
```

Deterministic Verdicts:

- `BUG_PRESENT` : Target source file contains the exact unpatched code pattern or cleanly applicable bug signature.
- `ALREADY_FIXED` : Target source already contains the upstream fix or equivalent logic from previous commits.
- `NOT_APPLICABLE` : Commit modifies files or subsystems that do not exist in Tortoise-WoW.
- `TURTLE_INTENTIONAL_DIVERGENCE` : Target code intentionally diverges from vanilla (e.g., custom race handling, debuff engine).
- `UNCERTAIN` : Context has diverged significantly; requires manual or advisory AI review.

15. Database Safety, Schema Catalog & Provenance

Database migrations undergo rigorous automated static analysis against `config/schema_catalog.json` (413 verified tables):

```
task.ps1 3 "sql/database_updates/world/20260507165648_world.sql"
```

Enforced Rules:

1. **Forbidden Progressive Columns**: Rejects any statement containing `patch` , `build` , `min_patch` , or `max_patch` .
2. **Custom Spell Boundary Protection**: `spell_template` IDs ≥ 40000 are reserved for Turtle-WoW custom spells. Modifications to these IDs trigger a validation failure.
3. **Custom World Entity Protection**: `creature_template` , `item_template` , `quest_template` , and `gameobject_template` IDs ≥ 300000 are reserved for Turtle custom content.
4. **Statement Anchoring**: Parser anchors table matching (`(?m)^\s` *) to prevent false positives from quest text strings like* "update my count"*.
5. **Entity Provenance Tracking**: Records source donor revision, target entity ID, original value, proposed value, and confidence rating in structured dossiers.

Database Scalper, Historic DB & Web Viewer Integration (`task scalp` / `task dashboard`)

The database scalper engine (`tools/porting/Extract-DbEntity.ps1`) coordinates a 3-tier database hierarchy for exhaustive entity analysis:

1. **Main / Authoritative**: Turtle-WoW base catalog (`tortoise-wow/sql/base/tw_world_<table_name>.sql`).
2. **Main Historic DB**: Brotalnia `brotalnia/database` (`reference-upstreams/lights-hope-database-history/world_full_14_june_2021.sql` from `world_full_14_june_2021.7z`) for original vanilla baseline values that Turtle-WoW did not change.
3. **Backup Donor DB**: `vmangos/core db_latest` (`reference-upstreams/vmangos-core/db_latest/mysql-dump/mangos.sql`) for updated vanilla definitions and constraints.
4. **Interactive Dashboard & REST API**: `tortoise-db-viewer` (`https://xian55.github.io/tortoise-db-viewer/` and `https://api.tortoiseclothing.org`) for live 3D models, tooltips, and drop tables.

```
# Scalp entity and display side-by-side diff against Turtle base schema
task.ps1 scalp item 19019 -Diff

# Export sanitized REPLACE INTO SQL directly to tools/queue/staging_sql/
task.ps1 scalp item 19019 -ExportSql

# Launch interactive AoWoW-style web dashboard
task.ps1 dashboard 19019
```

- **Instant Tooltip & Catalog Lookup**: Fetches parsed JSON records from `https://api.tortoiseclothing.org` (`/i/<id>` , `/n/<id>` , `/s/<id>` , `/q/<id>`) without needing a running MySQL server.
- **Authentic Vanilla Verification**: Cross-references Brotalnia's uncompressed 2021 snapshot to verify historical vanilla attributes.

- **Vanilla vs Turtle ID Tracking**: Leverages `tortoise-db-viewer/scripts/data/vanilla-ids.json` to verify whether an entity is authentic vanilla (2,430 items, 52 creatures) or Turtle-modified.
- **Automated Progressive Column Stripper**: Automatically strips progressive versioning columns (`patch` , `patch_min` , `patch_max` , `build`) and safeguards custom Turtle columns.

---

16. DBC, Client & Core Parity Auditor

The parity auditor (`ParityAuditor.ps1`) parses binary WDBC client data from `reference-upstreams/client-data-1.18.1/dbc` and verifies server constants:

```
task.ps1 parity
```

- **ChrRaces.dbc**: Verifies playable race records against `#define MAX_RACES 11` in `SharedDefines.h` .
- **Map.dbc**: Audits 57 map definitions against server map enum declarations.
- **Spell.dbc**: Verifies custom spell entry boundary rules.
- **ItemClass.dbc**: Verifies item classification bitmasks.

---

17. Crash Dump & Server Log Triage

Automated triage engine (`TriageEngine.ps1`) categorizes runtime issues and crash dumps across 17 distinct failure categories:

```
# Triage server startup or runtime log
task.ps1 triage "tortoise-wow/bin/Release/Server.log"

# Triage crash dump or assertion trace
task.ps1 crash "tortoise-wow/bin/Release/crash.dmp"
```

Supported Failure Classifications:

`ASSERTION_FAILURE` , `SEGMENTATION_FAULT` , `DATABASE_ERROR` , `DBC_ERROR` , `OPCODE_ERROR` , `MAP_LOAD_ERROR` , `SPELL_ERROR` , `SCRIPT_ERROR` , `NETWORK_ERROR` , `CONFIG_ERROR` , `AUTH_ERROR` , `MEMORY_LEAK` , `DEADLOCK` , `STARTUP_TIMEOUT` , `SHUTDOWN_HANG` , `CUSTOM_SYSTEM_ERROR` , `UNKNOWN_FAILURE` .

---

18. Worktree Isolation & Cleanup Management

All candidate modifications are strictly isolated to Git worktrees:

```
# View all managed active worktrees
task.ps1 worktree

# Safe cleanup of merged or obsolete worktrees
task.ps1 worktree-cleanup
```

Isolation Rules:

- Located exclusively in `<TargetRepo>/.worktrees/PORT-XXXX/` .
- Rooted on ephemeral branch `port/PORT-XXXX-<sha>` .
- Pruned cleanly using `git worktree remove` upon pipeline completion or rejection.
- Safety check prevents deletion of any directory outside `.worktrees/` .

---

19. Run IDs, Idempotency & Resumption

Every pipeline execution generates a unique, sortable Run ID:

```
RUN-yyyyMMdd-HHmmss-xxxx  (e.g., RUN-20260909-144022-7a1b)
```

- **Idempotent Resumption**: State store (`tools/state/state_store.json`) tracks active runs and stage transitions.
- Interrupted runs resume from the last certified state without re-running deterministic proofs.

---

20. Canonical 30-State Machine

Candidates transition through a strictly guarded 30-state lifecycle:

State	Category	Description	Allowed Next States
`DISCOVERED`	Initial	Candidate identified from upstream donor commit	`TRIAGED`, `BUG_PROOF_PENDING`, `ALREADY_FIXED`, `NOT_APPLICABLE`, `REJECTED`
`TRIAGED`	Priority	Categorized and scored by Priority Engine	`BUG_PROOF_PENDING`, `ALREADY_FIXED`, `NOT_APPLICABLE`, `TURTLE_INTENTIONAL_DIVERGENCE`, `REJECTED`
`BUG_PROOF_PENDING`	Proving	Undergoing deterministic diff & AST analysis	`BUG_PRESENT`, `ALREADY_FIXED`, `NOT_APPLICABLE`, `TURTLE_INTENTIONAL_DIVERGENCE`, `UNCERTAIN`
`BUG_PRESENT`	Verified Defect	Bug proved present in target source	`DEPENDENCY_PENDING`, `DEPENDENCY_READY`, `DB_CHECK_PENDING`, `ADAPTATION_REQUIRED`, `PATCH_READY`
`ALREADY_FIXED`	Terminal Clean	Upstream fix is already present in target	`NEEDS_REAUDIT`
`NOT_APPLICABLE`	Terminal Clean	System/file not present in target architecture	`NEEDS_REAUDIT`
`TURTLE_INTENTIONAL_DIVERGENCE`	Guarded	Target intentionally behaves differently	`HUMAN_REVIEW_REQUIRED`, `REJECTED`, `NEEDS_REAUDIT`
`UNCERTAIN`	Ambiguous	Cannot prove deterministically	`HUMAN_REVIEW_REQUIRED`, `ADAPTATION_REQUIRED`, `REJECTED`
`DEPENDENCY_PENDING`	Dependency	Waiting for predecessor commit to be ported	`DEPENDENCY_READY`, `HUMAN_REVIEW_REQUIRED`, `REJECTED`
`DEPENDENCY_READY`	Dependency	All required predecessor commits satisfied	`DB_CHECK_PENDING`, `ADAPTATION_REQUIRED`, `PATCH_READY`
`DB_CHECK_PENDING`	Database	Undergoing schema catalog and column audit	`DB_CHECK_PASS`, `DB_CHECK_FAIL`
`DB_CHECK_PASS`	Database	Schema valid, no forbidden columns, no collisions	`ADAPTATION_REQUIRED`, `PATCH_READY`
`DB_CHECK_FAIL`	Database	Invariant violation or column collision detected	`ADAPTATION_REQUIRED`, `HUMAN_REVIEW_REQUIRED`, `REJECTED`
`ADAPTATION_REQUIRED`	Adaptation	Requires code adjustment for Turtle custom systems	`PATCH_READY`, `HUMAN_REVIEW_REQUIRED`, `REJECTED`
`PATCH_READY`	Staged	Patch cleanly formatted and ready for isolated build	`COMPILE_PENDING`, `REJECTED`
`COMPILE_PENDING`	Build	Worktree created; queued for compilation	`COMPILE_PASS`, `COMPILE_FAIL`, `BLOCKED_BY_BASELINE`
`COMPILE_PASS`	Build	MSVC compilation succeeded with 0 errors	`STARTUP_PENDING`, `RUNTIME_PENDING`, `COMPLETE`
`COMPILE_FAIL`	Build	MSVC compilation failed	`BLOCKED_BY_BASELINE`, `ADAPTATION_REQUIRED`, `HUMAN_REVIEW_REQUIRED`, `REJECTED`
`STARTUP_PENDING`	Smoke Test	Binary launched in disposable environment	`STARTUP_PASS`, `STARTUP_FAIL`, `BLOCKED_BY_BASELINE`
`STARTUP_PASS`	Smoke Test	Binary initialized cleanly without crashes	`RUNTIME_PENDING`, `COMPLETE`, `HUMAN_REVIEW_REQUIRED`
`STARTUP_FAIL`	Smoke Test	Binary crashed or asserted on startup	`BLOCKED_BY_BASELINE`, `ADAPTATION_REQUIRED`, `REJECTED`
`RUNTIME_PENDING`	Runtime	Queued for in-game reproducer verification	`RUNTIME_PASS`, `RUNTIME_FAIL`
`RUNTIME_PASS`	Runtime	In-game behavior matches vanilla specification	`COMPLETE`, `HUMAN_REVIEW_REQUIRED`
`RUNTIME_FAIL`	Runtime	In-game reproducer failed	`ADAPTATION_REQUIRED`, `HUMAN_REVIEW_REQUIRED`, `REJECTED`
`HUMAN_REVIEW_REQUIRED`	Approval Gate	Ambiguous fix flagged for human decision	`HUMAN_APPROVED`, `HUMAN_REJECTED`
`HUMAN_APPROVED`	Approval Gate	Human operator certified fix for inclusion	`PATCH_READY`, `COMPILE_PENDING`, `COMPLETE`
`HUMAN_REJECTED`	Approval Gate	Human operator rejected fix	`REJECTED`
`BLOCKED_BY_BASELINE`	Baseline Block	Blocked because unchanged target HEAD is broken	`NEEDS_REAUDIT`, `REJECTED`
`NEEDS_REAUDIT`	Invalidation	Invalidated due to target HEAD moving or staleness	`BUG_PROOF_PENDING`, `TRIAGED`, `DISCOVERED`

State	Category	Description	Allowed Next States
`COMPLETE`	Final Pass	Certified, tested, committed to candidate branch	`NEEDS_REAUDIT`
`REJECTED`	Final Fail	Rejected candidate permanently archived	`NEEDS_REAUDIT`

---

21. CI / GitHub Actions PR Workflows

Two GitHub Actions workflows automate continuous integration across orchestration tools and candidate server builds:

1. `.github/workflows/orchestration-ci.yml` :
   - Runs on every push and pull request touching `tools/` , `config/` , or `docs/` .
   - Executes `task test` (Pester 38-case test suite).
   - Validates JSON schema contracts (`config/schemas/`).
   - Runs configuration health check (`task config-check`).
   - Verifies command reference and PDF generation.
2. `.github/workflows/server-candidate-ci.yml` :
   - Triggered when candidate port branches (`port/*`) are pushed.
   - Sets up MSVC 2022 toolchain and Ninja build system.
   - Runs baseline validation against target base commit.
   - Executes patch-aware compilation profile.
   - Uploads build logs and triage reports as artifacts.

---

22. Release Checkpoints & Manifest Generation

Release commands provide certification gates before tagging or publishing server releases:

```
# Evaluate release readiness gates
task.ps1 release-check

# Generate certified release manifest
task.ps1 release-manifest "Release-1.18.1-Update1"

# Create signed git tag (aborts if readiness gates fail)
task.ps1 tag-release "v1.18.1-update1"
```

- Manifests are saved to `docs/releases/<name>.json` with full commit SHAs, toolchain info, and test certifications.

---

23. Critical Safety Warnings

> **CAUTION**
> 1. **Single-Writer Constraint**: Never invoke `-AutoBuild` or compiler tasks concurrently in multiple shells. 2. **Never Commit Directly to Extended Main**: Candidate fixes must be isolated to candidate branches (`port/PORT-XXXX-<sha>`). 3. **Never Touch Working Tree**: Never execute `git checkout .` , `git reset --hard` , or `git clean -fd` in `tortoise-wow` . 4. **No Secrets in Logs**: Never print or commit API keys, authentication tokens, or private user passwords.

---

24. Troubleshooting & Common Failure States

Error Code / Symptom	Root Cause	Solution
`EXIT CODE 1: MAX_RACES invariant violation`	Candidate patch alters `MAX_RACES` or loop bound.	Restore `#define MAX_RACES 11` in patch; adapt race array allocations.
`EXIT CODE 1: Custom ID boundary collision`	Patch uses `spell_template` entry ≥ 40000 or world entry ≥ 300000.	Renumber entity to vanilla range (< 40000 or < 300000) or scalp correct ID.
`EXIT CODE 1: Target repository has uncommitted changes`	Working tree is dirty; worktree isolation safety triggered.	Commit or stash changes in `tortoise-wow` before running porting commands.
`EXIT CODE 2: Microsoft Edge not found`	Edge binary missing at standard 32/64-bit location.	Verify installation of Edge or update path in `Export-CommandReferencePdf.ps1` .

Error Code / Symptom	Root Cause	Solution
`EXIT CODE 2: CMake executable not found`	`cmake.exe` not detected in PATH or vcpkg tools directory.	Run `task config-check` to verify auto-discovery or update `config/twow-project.json`.
`STALE PACKAGE: Target base SHA mismatch`	Target repo HEAD advanced since package was staged.	Re-audit candidate via `task port <sha>` against new target HEAD.

25. Expected Status & Verdict Reference Values

Category	Canonical Allowed Values	Meaning
Pipeline Status	`PASS`, `FAIL`, `WARN`, `SKIPPED`, `BLOCKED`	Stage execution result code
Bug Verdict	`BUG_PRESENT`, `ALREADY_FIXED`, `NOT_APPLICABLE`, `TURTLE_INTENTIONAL_DIVERGENCE`, `UNCERTAIN`	Bug prover determination
Verification Mode	`Fast`, `Normal`, `Deep`	Verification intensity level
Build Profile	`world`, `auth`, `sql-only`, `playerbots`, `docs-only`	Selected MSVC build target
AI Usage	`NONE`, `ADVISORY_ONLY`, `AMBIGUITY_SYNTHESIS`	AI role in candidate evaluation
Exit Codes	`0` (PASS), `1` (VALIDATION_FAIL), `2` (TOOL_FAIL), `3` (SCHEMA_FAIL)	Process return codes

26. Architectural Strengths & Engineering Guarantees

The build and porting automation architecture provides a high-assurance, non-destructive, enterprise-grade engineering framework:

Architectural Dimension	Modern Autonomous Implementation	Tangible Engineering Guarantee
Git Working Tree Safety	Strict Git worktree isolation in ephemeral `.worktrees/candidate-PORT-XXXX/`.	Target repo checkout is **100% clean and immune** to accidental corruption or debris.
Rollback & Cleanup	Non-destructive: simply unlinks the worktree (`Remove-IsolatedWorktree`).	Eliminates catastrophic data loss risk; never discards uncommitted work.
Commit Staging & Branches	Changes committed exclusively to candidate branches (`port/PORT-XXXX-<sha>`).	Enables clean PR reviews, branch audits, and CI smoke testing before integration.
Bug Existence Verification	Deterministic Bug Prover (`BugProver.ps1` / `task prove`).	Classifies `BUG_PRESENT`, `ALREADY_FIXED`, `NOT_APPLICABLE`, or `TURTLE_DIVERGENCE` before touching code.
Build Efficiency	Patch-Aware Build Profiles (`world`, `auth`, `sql-only`, `docs-only`).	Bypasses compilation for SQL/docs (0s), targets single daemons (15-45s), maximizing iteration speed.
Database Migration Safety	Cached 413-table schema catalog (`DbAuditor.ps1`) and strict boundary guards.	Blocks forbidden progressive columns (`patch`, `build`) and protects custom ranges (`spell` $\ge 40k$, world $\ge 300k$).
Client Data Parity	Automated binary WDBC parser (`ParityAuditor.ps1` / `task parity`).	Guarantees code parity with 1.18.1 client DBCs (`MAX_RACES = 11`, maps, spell definitions).
State Tracking & Resumption	Atomic canonical JSON state store (`state_store.json`) with 30-state transition guards.	Deterministic run IDs (`RUN-yyyyMMdd-HHmmss-xxxx`), transition guards, and resume capability.
AI Token Efficiency	Deterministic-first gating, bounded context (≤ 200 lines), composite SHA256 caching.	0 AI tokens spent on deterministic rejections; prevents hallucination via `ADVISORY_ONLY`.
Runtime Reliability	Disposable startup smoke tests (`SmokeTest.ps1`) and 17-category log triage.	Catches assertion failures, heap corruptions, and missing DBCs before candidate commits are certified.
Automated Testing & CI	Comprehensive 38-spec test suite (`task test`) + 2 GitHub Actions CI workflows.	Sub-7-second automated verification ensuring every invariant, schema, and command passes.

27. Daily Cheat Sheet & Top Fast Commands

This quick-reference cheat sheet summarizes the most frequent commands you will run on a day-to-day basis.

26.1. The Top Commands at a Glance

Command	Full Syntax	Primary Purpose	When to Use
Batch Auto-Pilot	`task auto-pilot 10`	Full autonomous pipeline: lore scouting, DB audit, AI context dossier, MSVC worktree compile, and candidate branch commit for 10 commits.	Your primary hands-free daily command for high-velocity porting.

Command	Full Syntax	Primary Purpose	When to Use
Tier-Filtered Auto-Pilot	`task auto-pilot 10 1`	Same end-to-end auto-pilot pipeline, but restricted strictly to Tier 1 (Crash, Exploit, and Critical Security fixes).	When focusing specifically on server stability, security, or crash elimination.
Batch Auto-Commit	`task port-batch 10 -AutoCommit`	Equivalent to `task auto-pilot 10`. Runs all research stages (lore, DB audit, AI context) and triggers the MSVC compile & commit gate.	Exact semantic alias for batch auto-pilot.
Single Auto-Port	`task auto-port <sha>`	One-command end-to-end port, compile, and commit for a single specific upstream commit SHA.	When investigating or porting a specific upstream commit SHA directly.
Refresh Upstream Roadmap	`task roadmap-refresh -FetchLatest`	Connects to upstream `vmangos/core`, fetches the latest commits, audits against Turtle core, recalculates priority scores, and updates `docs/ROADMAP.md`.	Weekly or when new upstream commits are released and you want to catch them.
Offline Roadmap Audit	`task roadmap-refresh`	Re-evaluates and ranks all cached candidate commits offline without network requests.	When re-ranking local candidates or updating queue files offline.
Compile Server	`task build` *(or `task build world`)*	Directly invokes CMake / MSVC 2022 to build the target server daemon (`world`, `auth`, `sql-only`, `playerbots`).	To verify current core builds cleanly without running the porting pipeline.
Run All Unit Tests	`task test`	Runs the universal 38-spec automated Pester test suite in under 7 seconds.	Before and after any major tooling or policy change.
Prove Bug Existence	`task prove <sha>`	Deterministically proves whether a bug exists in Tortoise-WoW Extended without modifying any code.	Quick triage to see if an upstream fix is already fixed or applicable.
Pipeline Status	`task status`	Outputs current target Git HEAD, active worktree count, staged ready packages, and active run IDs.	Anytime you want a rapid health check of the engineering environment.
Entity Scalp & Diff	`task scalp <table/type> <id> -Diff`	Extracts item, NPC, spell, or quest records via tortoise-db-viewer API/dataset, strips progressive columns, and generates sanitized SQL diff.	When fixing or verifying custom database entities against vanilla data.
Web DB Dashboard	`task dashboard [id]`	Launches the interactive AoWoW-style web database viewer and 3D model/tooltip dashboard.	For instant visual, tooltip, and drop-rate inspection of entities.
Worktree Cleanup	`task cleanup`	Safe, non-destructive disposal of ephemeral `.worktrees/candidate-*` directories.	Periodic cleanup after large batch runs; leaves working tree 100% clean.
Regenerate PDF & HTML	`task pdf`	Converts `COMMAND_REFERENCE.md` into cleanly formatted `COMMAND_REFERENCE.html` and publication-ready `COMMAND_REFERENCE.pdf`.	Whenever documentation or command specifications are updated.

26.2. Clean CLI Invocations (From Default Location / Any Prompt)

You do not need to change directory (`cd`) to run any of these commands. You can execute them directly from `C:\Users\Admin>` or any terminal prompt:

1. Autonomous Auto-Pilot & Batch Porting:

```
# Run 10 commits through full auto-pilot (natural priority)
& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" auto-pilot 10

# Run 10 Tier-1 (critical crash / exploit / security) commits through auto-pilot
& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" auto-pilot 10 1

# Run 10 Tier-2 (combat accuracy, spells, mechanics) commits through auto-pilot
& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" auto-pilot 10 2

# Port, compile, and commit a specific candidate commit end-to-end
& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" auto-port <sha>

# Batch port 10 commits with immediate compilation & commit (semantic equivalent to auto-pilot)
& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" port-batch 10 -AutoCommit

# Preview batch auto-pilot execution plan without touching code or worktrees
& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" auto-pilot 10 -DryRun
```

2. Upstream Research, Roadmap & Priority Ranking:

```
# Connect to upstream vmangos/core, fetch latest commits, and audit roadmap
& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" roadmap-refresh -FetchLatest

# Offline audit and re-ranking of all cached candidate commits
& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" roadmap-refresh

# Inspect the single next highest-priority candidate to investigate
& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" next

# Display 12-factor priority rankings across all candidate commits
& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" rank

# Search 22,155 indexed official forum threads for mechanics or lore
& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" 2 "<sha_or_topic>"
```

3. Build Toolchain, Verification & Testing Gates:

```
# Compile target server world daemon (mangosd.exe) via MSVC 2022
& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" build world

# Compile authentication daemon (realmd.exe) via MSVC 2022
& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" build auth

# Compile and commit all staged candidate packages in isolated worktrees (local only)
& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" build-packages

# Integrate and push all verified passing candidate fixes to GitHub extended branch
& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" push-extended

# Run the complete 38-spec automated orchestration test suite
& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" test

# Deterministically prove whether a bug exists before touching code
& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" prove <sha>

# Verify target repository baseline health and SHA pinning
& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" baseline

# Execute disposable server startup smoke test
& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" smoke
```

4. Database Scalping & Parity Auditing:

```
# Scalp entity definition via tortoise-db-viewer, strip progressive columns, and diff
& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" scalp item 19019 -Diff

# Open entity in official AoWoW web database dashboard
& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" dashboard 19019

# Audit pending database migration SQL against 413 cached table schemas
& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" db-audit

# Audit client binary DBC parity against server source code (MAX_RACES = 11)
& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" parity dbc
```

5. Workspace Status, Maintenance & Documentation:

```
# Run rapid pre-flight audit before starting work (~1.7s)
& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" system-check

# Run full deep system certification before major releases (~10.5s)
& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" system-check full

# Display live pipeline state, target Git HEAD, and active run IDs
& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" status

# Cleanly dispose of ephemeral candidate worktrees in .worktrees/
& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" cleanup

# Regenerate COMMAND_REFERENCE HTML and printable PDF
& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" pdf
```

27.3. Target Repository Authority & Local Workspace Guarantees

> IMPORTANT
> **Repository Authority & Target Server**: - **Sole Server Target**: All ported fixes, investigations, and compiled commits target `Ildourol/tortoise-wow-extended` exclusively. - **Local Toolchain Execution**: All automation scripts, analysis engines, and state trackers operate strictly locally on this machine with zero remote orchestration repository connections.